



Průmyslová robotika II – Semestrální projekt

Integrace osvětlení robota klavíristy

Datum odevzdání: 7.12.2025

Vypracoval: Dalibor Slíva SLI0123, Jan Besta BES0057, František Kaštovský KAS0249

Obsah

Seznam obrázků.....	2
Seznam tabulek	3
1 Úvod do problematiky – Rozbor zadání.....	4
2 Raspberry Pi.....	5
2.1 Backend.....	5
2.1.1 WebSocket.....	5
2.1.2 HTTP REST API.....	6
2.1.3 Služby běžící na pozadí	6
2.2 C++ Aplikace.....	7
3 KUKA robot.....	8
4 Komunikace s KUKA robotem.....	9
4.1 Proměnné v robotovi	9
5 LED osvětlení	10
5.1 Hardware.....	10
5.2 Montáž	11
6 Závěr – zhodnocení výsledků.....	17
7 Zdroje.....	18

Seznam obrázků

Obrázek 1 - Blokové schéma systému	4
Obrázek 2 – Elektrické schéma zapojení LED pásků a jejich ovládání pomocí RPI	10
Obrázek 3 – Čtyřkanálový logic level shifter (převodník logické úrovně)	11
Obrázek 4 – 3D model levé podstavy pro klávesy	11
Obrázek 5 – Levá strana montáže podstavy pod klávesami	12
Obrázek – Kompletní montáž podstavy pod klávesami.....	12
Obrázek - 3D model příchytky pro NEON LED pásek.....	13
Obrázek - 3D model příchytky na hliníkový "L" profil především pro spodní LED pásek.....	13
Obrázek - 3D model rohu spodního uchycení hliníkového "L" profilu a LED pásku.....	14
Obrázek - 3D model koncovky pro spodní konstrukci.....	14
Obrázek – Finální podoba instalované konstrukce pro podsvícení.....	15
Obrázek 12 a 13 – Krabíčka pro logic level shifter	15
Obrázek - 3D model držáku pro router.....	16
Obrázek - 3D model krabíčky pro Raspberry Pi	16
Obrázek - 3D model krabíčky pro step-down měniče.....	16
Obrázek 17 – Finální podoba rozmístění komponent.....	17

Seznam tabulek

Tab. 1 - Proměnné komunikující s robotem	9
Tab. - Seznam použitých pinů RPI 4	11

1 Úvod do problematiky – Rozbor zadání

Robot klavírista disponuje dvojím osvětlením. První osvětlení je ze spodní strany celého systému a umožňuje uživateli zobrazovat vnitřní stavy systému. Druhé osvětlení je pod samotným piánem/klaviaturou a slouží pro vizualizaci zmáčknuté klávesy spolu s vizuálním efektem.

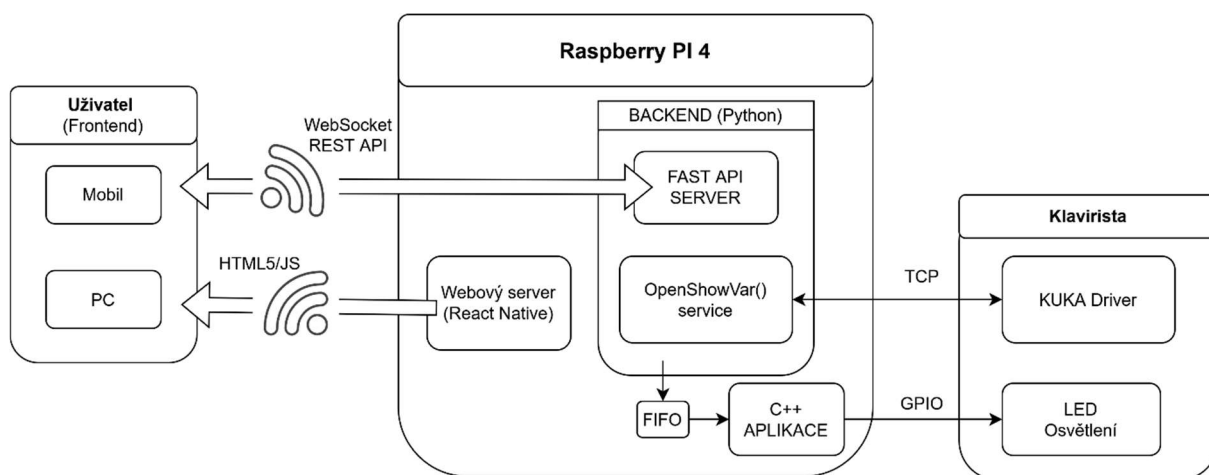
Celý systém je postaven na Raspberry Pi 4, které funguje jako centrální řídicí jednotka. Na Raspberry Pi běží backendová aplikace v Pythonu, poskytující REST API a zajišťující komunikaci mezi uživatelem a fyzickým systémem „Klavírista“. Uživatel přistupuje k systému buď z mobilního, nebo počítačového zařízení, a to prostřednictvím webového rozhraní nebo aplikace postavené na React Native. Frontend odesílá požadavky přes síťové rozhraní k backendu (REST API / HTTP), případně získává stavové informace opět ze serveru běžícího na Raspberry Pi.

Backend je rozdělen na dvě hlavní části. První část tvoří FASTAPI server, který zpracovává příchozí požadavky uživatele, provádí validaci, plánování akcí a poskytuje odpovědi zpět do frontendu. Druhou část backendu tvoří služba OpenShowVar(), která zajišťuje online komunikaci mezi Raspberry Pi a řídicími moduly robota KUKA. Backend podle potřeby předává instrukce do robota s využitím knihovny OpenShowVar díky, které pak následně komunikuje s robotem nebo dalšími podsystémy.

Pro komunikaci s hardwarem je v systému přítomna také C++ aplikace, která běží na Raspberry Pi a je propojena s backendem prostřednictvím FIFO sdílené paměti. C++ aplikace vykonává časově kritické nebo hardwarové operace, jako je ovládání externího LED osvětlení či doplňkových efektů.

Na straně zařízení klavírista se nachází dva hlavní moduly:

- KUKA Driver řídí pohyby samotného robotického ramene na základě dat přijímaných přes OpenShowVar.
- LED Osvětlení je samostatně řízené pomocí C++ aplikace na Raspberry Pi.



Obrázek 1 - Blokové schéma systému

2 Raspberry Pi

Raspberry Pi je jedna z nejdůležitějších komponent v projektu, v našem případě se jedná o verzi Raspberry Pi 4 s 4 GB vnitřní paměti. Obohatili jsme jej o potřebné chladiče kvůli chlazení a zavřeli jej do krabičky v rámci umístění na celém systému. Je to opěrný bod, kde se potkává hardware se softwarem. Je zde realizována celá funkčnost komunikace, ovládání LED pásků a hostování webového serveru.

2.1 Backend

Backend je postaven na asynchronním frameworku FastAPI, který zajišťuje současný provoz REST API, WebSocket serveru a paralelně běžících interních služeb pro komunikaci s robotem KUKA pomocí knihovny OpenShowVar. Celá architektura je navržena modulárně a skládá se z několika vrstev. První z nich představuje REST API, které poskytuje synchronní přístup k funkcím robota, jako jsou dotazy na stav, připojení či odpojení nebo spouštění jednotlivých režimů. Druhou vrstvou tvoří WebSocket rozhraní, které umožňuje obousměrnou komunikaci v reálném čase s Frontendem a pracuje s konceptem virtuálních místností. Třetí vrstvou představují interní asynchronní služby, které běží na pozadí a starají se o dohled nad stavem robota a zjišťují jeho aktuální polohu. V závislosti na poloze se pak odesílají data do C++ aplikace.

2.1.1 WebSocket

Správa klientů a místností

Systém obsahuje vlastní logiku pro správu více současně připojených klientů pomocí WebSocket protokolu. Klienti jsou organizováni do tzv. místností, což umožňuje oddělenou komunikaci podle účelu nebo typu služby. Každému připojení jsou přiřazeny základní informace o zařízení, identifikátor a jeho role v systému. Backend si vede přehled o všech aktivních klientech a umožňuje rychle rozesílat zprávy celé skupině nebo jen vybraným členům.

Role performer a watcher

Systém rozlišuje dva typy připojení. Prvním je performer, což je aktivní klient, který ovládá robota, generuje signály `note_on` či `note_off` a může také spouštět skladby. Druhým typem je watcher, který komunikaci pouze sleduje a dostává aktualizace o stavu, přehrávaných notách či dalších událostech. RoomHub zajišťuje přísnou kontrolu nad přidělováním rolí a poskytuje REST endpointy, které umožňují externě zobrazit seznam aktuálních performerů a watcherů.

Životní cyklus spojení

Po navázání WebSocket spojení je klient zaregistrován do příslušné místnosti a začne komunikovat se serverem pomocí předem definovaného formátu zpráv. Během komunikace klient odesílá zprávy ve formátu JSON, které backend interpretuje podle typu události. Backend následně provede odpovídající akci, například předání `note_on` signálu do služby robota nebo broadcast zprávy pro ostatní klienty. Při odpojení nebo chybovém stavu je klient ze systému odstraněn a případná navázaná logika se korektně ukončí.

Watchdog aktivity klientů

Součástí WebSocket logiky je časovač sledující aktivitu jednotlivých klientů. Pokud klient neodešle po dobu 15 sekund žádný signál, je označen jako neaktivní a může být automaticky odpojen nebo být zbaven role performerera.

2.1.2 HTTP REST API

Stav a ovládání robota

Backend poskytuje implementaci standardního HTTP API, které slouží především pro získávání diagnostických informací a spouštění konkrétních funkcí systému. Klient může například zjistit aktuální stav robotického zařízení, získat informace o dostupných režimech nebo ověřit, zda je robot připraven na komunikaci. Odpovědi jsou strukturované tak, aby umožňovaly jednoduché využití na frontendu. Kromě pasivního získávání informací umožňuje API také spouštět určité akce. Sem může patřit navázání spojení s robotem, aktivace či deaktivace některého provozního režimu nebo odeslání hodnoty proměnné do robota.

Správa připojených klientů

HTTP rozhraní poskytuje rovněž přístup k informacím o WebSocket klientech. Umožňuje zobrazit všechny aktivní klienty, zjistit jejich role a případně provádět administrativní zásahy, jako je odpojení určité skupiny.

2.1.3 Služby běžící na pozadí

Asynchronní komunikace s robotem

Základní službou na pozadí je komponenta, která udržuje síťové spojení s robotickým zařízením a zajišťuje bezpečné čtení a zápis řídicích proměnných. Převádí požadavky aplikace na konkrétní pohyby a akce robota a využívá zámky, opakované pokusy a kontrolu chyb, aby byla komunikace stabilní a plynulá i při vyšší zátěži nebo dočasné nedostupnosti.

Periodické úlohy

Další částí jsou periodické smyčky, které pravidelně ověřují dostupnost robota, automaticky obnovují připojení a průběžně načítají jeho aktuální stav do interní cache. Tím se snižuje počet přímých dotazů na robota a zvyšuje celková rychlost odezvy systému.

Komunikace přes FIFO s C++ aplikací

Specifická smyčka zajišťuje propojení s externím C++ aplikací pomocí dvou FIFO rour. Na základě polohy robota rozpoznává stisk klávesy nebo přehrávání skladby a odesílá odpovídající číselné hodnoty do datové roury. Druhá roura slouží k příjmu potvrzení, že zpráva byla zpracována. Pokud potvrzení nedorazí, zpráva se znovu odešle. Tento mechanismus zajišťuje spolehlivou výměnu dat mezi Python backendem a C++ částí bez blokování hlavní asynchronní smyčky.

2.2 C++ Aplikace

Pro řízení adresovatelných LED pásků přes GPIO piny Raspberry Pi byla použita knihovna `rpi_ws281x`, která představuje obdobu knihovny FastLED běžně využívané na mikrokontrolérech, jako je Arduino nebo ESP32. Jádrem programu je pro horní LED pásek umístěný pod klávesami i pro spodní pásek sloužící k podsvícení totožné. V obou C++ aplikacích je implementována komunikace s backendem pomocí FIFO, konkrétně pomocí `named pipe`, což je speciální soubor v Linuxu umožňující předávání dat mezi procesy. Tato jednosměrná bajtová fronta slouží k předání číselné hodnoty získané backendem od robota, na jejímž základě je spuštěna příslušná animace LED pásků. Celkem jsou FIFO čtyři, dva páry slouží pro posílání hodnot do C++ aplikací a druhé dvě slouží pro potvrzení informace backendu, že bylo číslo přijato a úspěšně dekodováno (horní a spodní zvlášť). Oba pásky pracují ve dvou režimech: aktivním a pasivním. Aktivní režim reaguje na činnost robota, tedy konkrétně na to, jakou klávesu robot stiskne. Pasivní režim je určen pro přehrávání několika předdefinovaných efektů ve smyčce v okamžiku, kdy je robot po určité době nečinný. Tato prodleva, označující dobu do přepnutí do standby režimu, je konfigurovatelná.

V aktivním režimu horního LED pásku je spuštěna animace dvou hadů, kteří se šíří z místa odpovídajícího stisknutí klávese na obě strany, přičemž barva obou hadů je vybírána náhodně. U spodního LED pásku je aktivní režim založen na efektu světelného nádechu s následným časově řízeným dozíváním. Barva tohoto efektu je odvozena od hodnoty čísla, které robot předá při stisku klávesy, a tedy i od hudebního tónu. Platí, že čím nižší je číslo odpovídající tónu, tím hlubší a tmavší je výsledná barva nebo její odstín.

3 KUKA robot

Program v robotu KUKA vychází z velké části z původního programu bakalářské práce Adama Bátorly. Struktura programu se skládá z programu MAIN_Klavirista_V2, který běží ve smyčce a čeká na ovládání uživatelem. Z uživatelského rozhraní můžou přijít dva pokyny, buď zapnout některou z před nahraných písní nebo přejít do režimu stínování. Když se zapne píseň tak program spustí funkci, která postupně volá funkci PLAY() s parametry čísla klávesy a délky hraní. V této funkci pak robot podle parametrů najede nad klávesu díky startovací pozici a posunu, který určuje číslo klávesy, zmáčkne klávesu a drží ji, dokud je časovač menší než parametr délky hraní, tímto způsobem se přehraje celá píseň.

V režimu stínování čeká robot, dokud z uživatelského rozhraní nepřijde proměnná s informací o zmáčknuté klávese, když tato proměnná nabude hodnoty větší než nula, program zavolá funkci PLAYRT() tato funkce je podobná jako funkce PLAY() a má stejné parametry. Hlavní rozdíl je v tom, že do této funkce vstupuje jako parametr PyNoteDuration, která vstupuje s velkou hodnotou, protože v době, kdy je tato funkce zavolána ještě uživatel nemusel pustit klávesu. Reálná hodnota PyNoteDuration tedy přijde až po puštění klávesy uživatelem.

4 Komunikace s KUKA robotem

Komunikace s robotem běží za pomoci knihovny OpenShowVar, která využívá TCP komunikace. OpenShowVar čte a zapisuje přímo do proměnných uložených v robotu, tudíž veškerou komunikaci obsluhuje program v Pythonu.

4.1 Proměnné v robotovi

Pro zpracování projektu bylo nutné vytvořit nové uživatelské proměnné v config souboru robota. Všechny tyto proměnné začínají písmeny Py aby bylo jasné odlišeno, že se jedná o komunikační proměnné s programem v pythonu. Niže je tabulka s proměnnými, jejich datovými typy a jestli jsou pro program v robotu vstupní nebo výstupní.

Tab. 1 - Proměnné komunikující s robotem

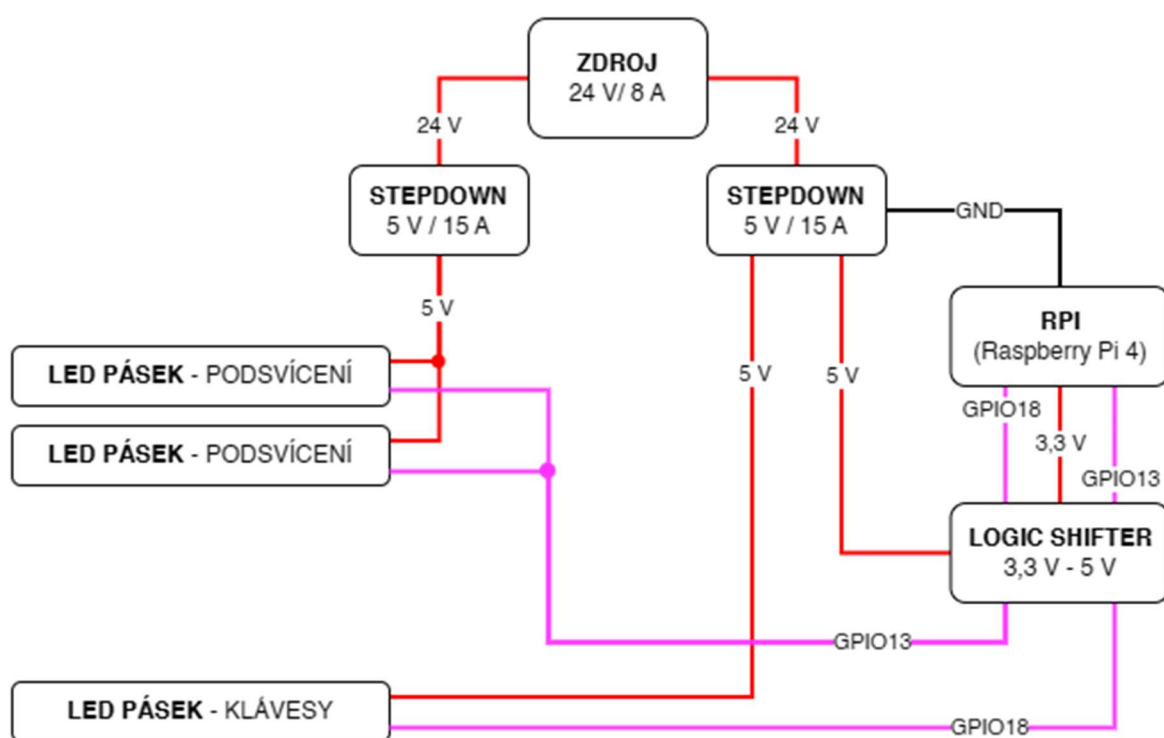
Proměnná	Návratová hodnota proměnné	Vstupy a výstupy	Datový typ
PyShadow	PyShadowFb	Vstup	Bool
PyNoteNumber		Vstup	Int
PyNoteDuration		Vstup	Int
PyGoToNote	PyGoToNoteFb	Vstup	Int
PyNotePlayed		Výstup	Int
PyKey		Výstup	Real
PyPlayingSong		Výstup	Bool
PyREGGAE		Vstup	Bool
PySTUPNICE		Vstup	Bool
PyBEETHOVEN		Vstup	Bool
PyMAX_VOLUME		Vstup	Bool
PyEND		Výstup	Bool
PyACK		Výstup	Bool
PyWait		Výstup	Bool
PySongNumber		Výstup	Int

5 LED osvětlení

Pro realizaci osvětlení byly použity adresovatelné RGB LED pásy typu WS2812B ve dvou variantách: 100 LED/m s krytím IP30, vložené do neonového silikonového profilu pro dosažení difuzního efektu u horního LED pásu umístěného pod klávesami, a 60 LED/m s krytím IP65 pro spodní LED pásek sloužící k podsvícení obvodu celé buňky.

Tyto LED pásy fungují na principu jednovodičového datového řetězce, kdy každá jednotlivá LED obsahuje vlastní řídicí čip. Ten si z příchozího sériového datového streamu přečte prvních 24 bitů (GRB) určených pro sebe a zbytek dat propouští dál. Adresování tedy vzniká automaticky podle fyzického pořadí LED v pásu, bez nutnosti jakékoliv konfigurace nebo nastavování adres. Po přijetí dat a reset pulsu si LED uloží barvu do svého PWM driveru a samostatně ji zobrazuje.

5.1 Hardware



Obrázek 2 – Elektrické schéma zapojení LED pásků a jejich ovládání pomocí RPI

Tyto LED pásy jsou však poměrně energeticky náročné. Při maximálním jasu všech tří RGB segmentů (bílá barva) odebírá jedna LED přibližně 60 mA. U pásu o délce 4,6 m a hustotě 60 LED/m to představuje proud až kolem 16,5 A.

Při návrhu napájení bylo nutné s tímto odběrem počítat, a proto byl jako hlavní zdroj využit již instalovaný 24 V / 8 A napájecí zdroj Siemens, který měl dostatečnou výkonovou rezervu. Z tohoto zdroje byl pro každou světelnou větev vyveden samostatný DC step-down měnič na 5 V s maximální špičkovou proudovou kapacitou až 20 A. Kromě samotného napájení LED pásků je nutné zajistit také společnou zem (GND) s řídicí jednotkou Raspberry Pi.



Obrázek 3 – Čtyřkanálový logic level shifter (převodník logické úrovně)

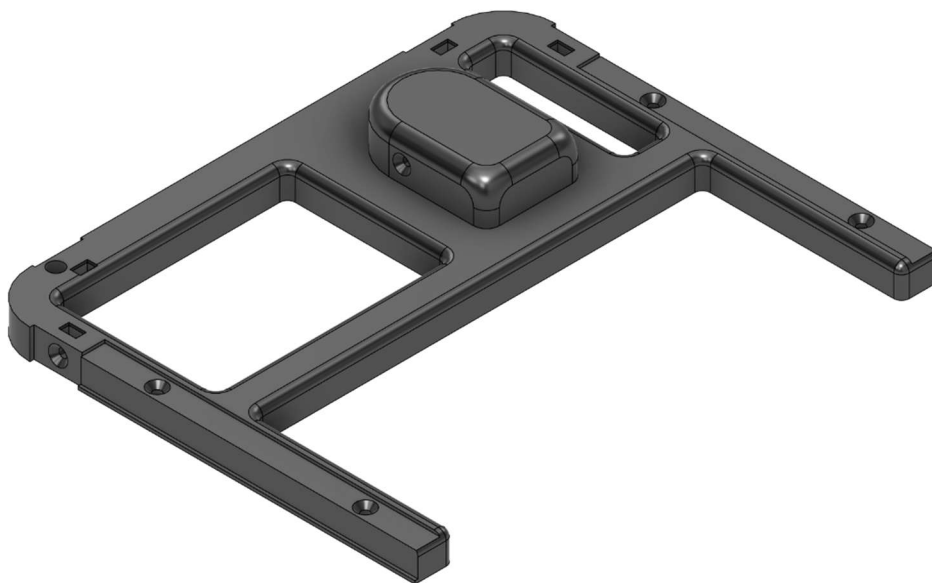
Právě u RPI se projevil problém způsobený délkou datového vodiče pro řízení LED pásků. Raspberry Pi pracuje s logickou úrovní 3,3 V, zatímco LED pásky vyžadují napájení 4,5–5,5 V a zároveň musí řídicí signál splňovat podmínku $V_{DATA} \geq 0,7 \cdot V_{CC}$. S rostoucí délkou vodiče docházelo k úbytku napětí a následně ke zkreslení zobrazení na LED pásku. Tento problém byl vyřešen zapojením logic level shifteru, který zvýšil logickou úroveň signálu z GPIO pinů z 3,3 V na 5 V a eliminoval tak problémy způsobené úbytkem napětí na dlouhém vedení. Pro ovládání LED pásků byly zvoleny GPIO18 (PIN 12) a GPIO13 (PIN 33), jelikož piny GPIO12 a GPIO18 mají společný kanál PWM0 a obdobně GPIO13 a GPIO19 kanál PWM0.

Tab. 2 - Seznam použitých pinů RPI 4

Název funkcionality	PIN
Řízení horního LED pásku	PIN 12 = GPIO18 (PWM0)
Řízení spodního LED pásku	PIN 33 = GPIO13 (PWM1)
Napájení (logic level shifter, společná GND)	PIN 1/39, PIN 14

5.2 Montáž

Pro namontování horního i spodního LED pásku bylo potřeba vytvořit specifické prvky. Nejlepší volbou byl tedy 3D tisk. Během vývoje vzniklo hned několik prototypů navržených 3D modelů, které byly postupně tištěny až byly vylazeny do požadované podoby a kvality.

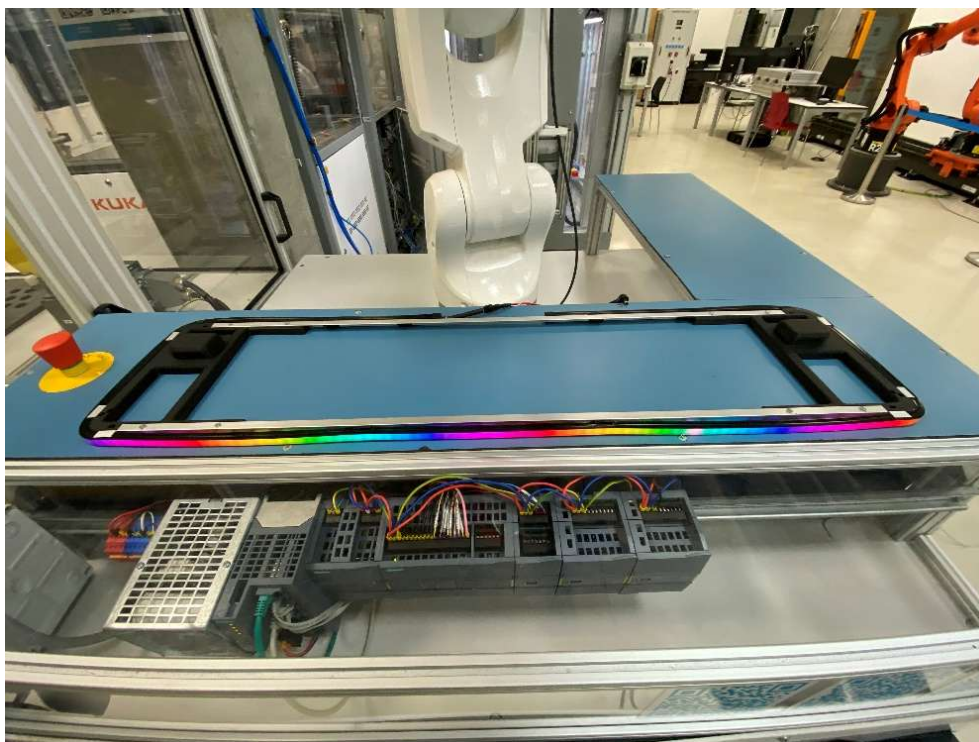


Obrázek 4 – 3D model levé podstavy pro klávesy

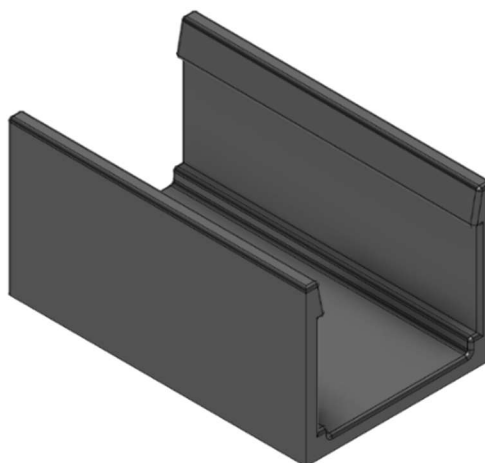
Pro vměštění horního LED pásu v neonovém obalu bylo nutné nadzvednout klávesy. Tohoto bylo docíleno díky vytvoření podstavy pod klávesy, která zajistila již zmíněný zdvih kláves, ale také usazení samotných kláves, aby zůstaly fixně na pozici. Identicky zrcadlově převrácena byla vyhotovena pravá část, která byla společně s levou spojena hliníkovými "L" profily pro zpevnění a zajištění zarovnání, které jsou k podstavám přišroubovány.



Obrázek 5 – Levá strana montáže podstavy pod klávesami

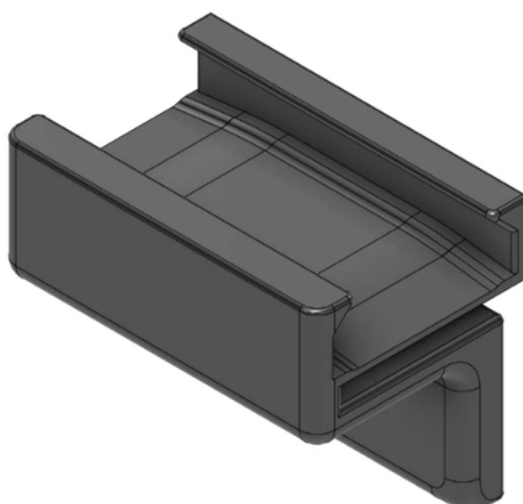


Obrázek 6 – Kompletní montáž podstavy pod klávesami



Obrázek 7 - 3D model příchytky pro NEON LED pásek

Rovněž přišroubovány jsou hliníkové i 3D tiskem zhotovené příchytky, které mají za úkol držet neonový LED pásek po obrysu 3D tisku podstavy.



Obrázek 8 - 3D model příchytky na hliníkový "L" profil především pro spodní LED pásek

Značné množství těchto příchytok slouží k nasazení na hliníkový "L" profil, kdy jsou na profilu pouze nasunuty, ale je kdykoli možné doplnit je o oboustrannou pásku, pro kterou slouží styčná plocha ze vnitřní strany profilu. Jsou určeny především pro zacvaknutí spodního LED pásku, který je zalitý v silikonu (IP65).



Obrázek 9 - 3D model rohu spodního uchycení hliníkového "L" profilu a LED pásku

Konstrukce spodního osvětlení (podsvícení) musela řešit jednak možnost přepravy celé buňky tak, aby při ní nebyl LED pásek poškozen, a jednak dostatečnou intenzitu osvětlení, jelikož je podlaha montážního prostoru buňky relativně vysoko. Z tohoto důvodu nemůže být na úplném kraji spodní desky či na nosném krajním profilu, a proto byl tento kus vymodelován pro přichycení vruty ze spodní strany do lepené desky a prostor pro zacvaknutí LED pásku je pod úhlem 30° pro docílení maximální efektivity osvětlení.



Obrázek 10 - 3D model koncovky pro spodní konstrukci

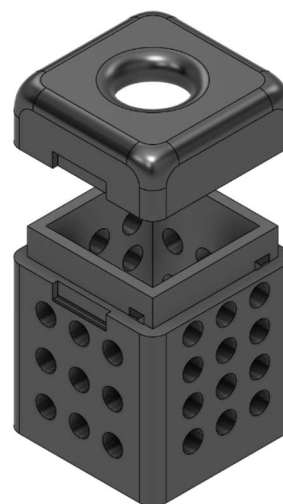
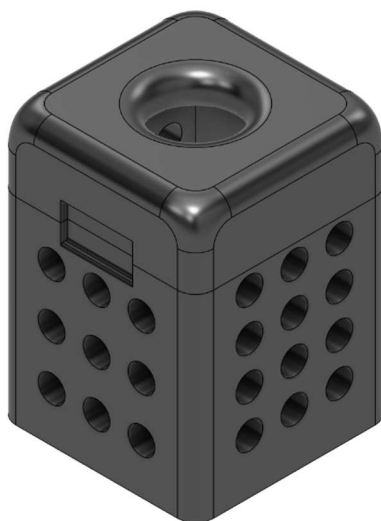
Výtisky v každém rohu jsou směrem ke střední příčce spojené s dalším konstrukčním prvkem, který pouze drží druhý konec hliníkové lišty, která mezi nimi tvoří propojení opět z důvodu zpevnění, a také správného zarovnání.



Obrázek 11 – Finální podoba instalované konstrukce pro podsvícení

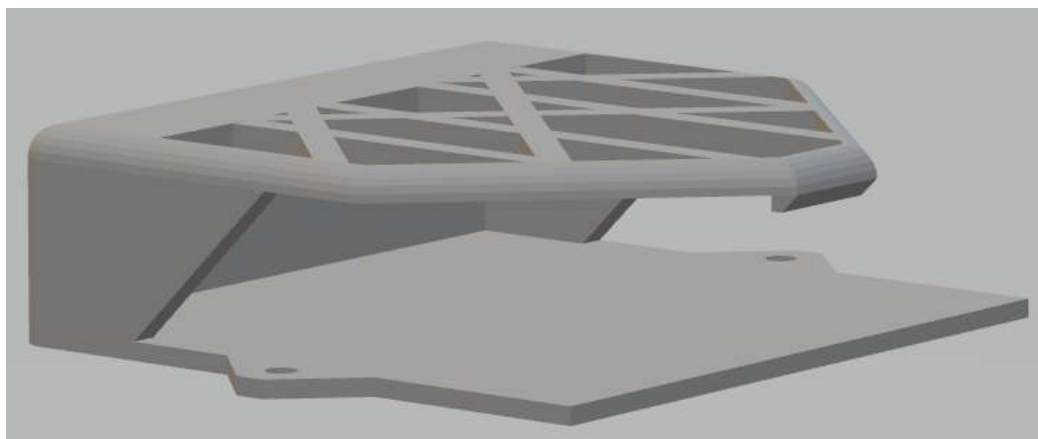
V rámci robota jsme pro podsvícení nainstalovali dvě konstrukce. Každou na jednu polovinu spodní desky oddělené podpurným profilem. Konstrukci jsme umístili přibližně deset centimetrů od kraje a společně s již zmíněným nakloněním dosahuje požadovaného efektu.

Jako instalační kryt pro logic level shifter byla vytvořena krabíčka, která je do konstrukční MDF desky přišroubována dvěma vruty. Samotná DPS je uvnitř lehce přilepená oboustrannou páskou kvůli absenci montážních otvorů.



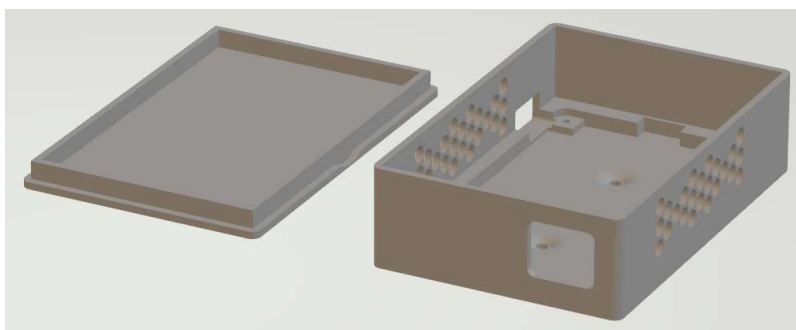
Obrázek 12 a 13 – Krabíčka pro logic level shifter

Poměrně velkým instalačním prvkem je držák routeru, který je rovněž přišroubován do MDF desky a zajišťuje bezpečné uchycení a zároveň přístup k jeho HW vstupům a výstupům.



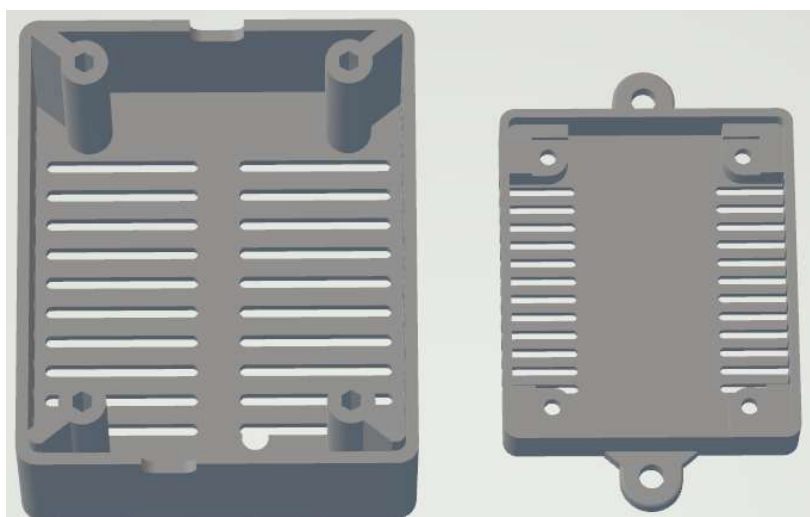
Obrázek 14 - 3D model držáku pro router

Prakticky totožný princip plní krabička pro RPI, kde je RPI uchyceno 4 vruty, a která je už uzavíratelná a disponuje pouze otvory pro vstup ethernetu RJ45, napájení USB-C a větrání.

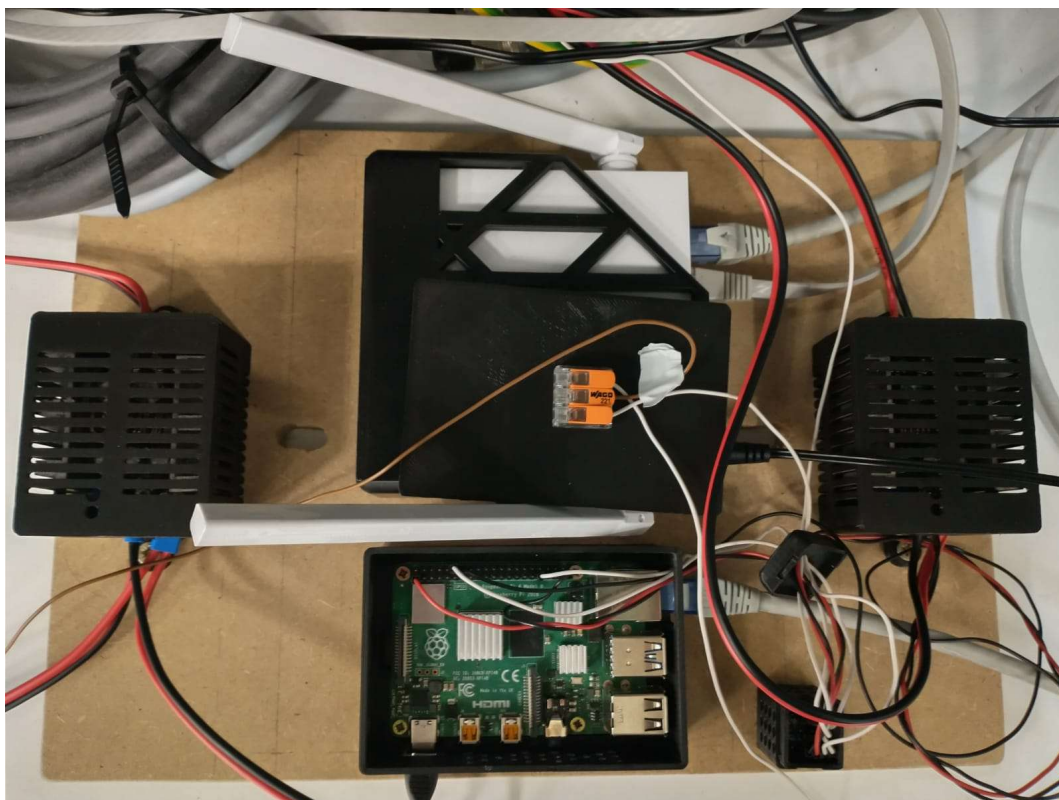


Obrázek 15 - 3D model krabičky pro Raspberry Pi

Ochranu před případným náhodným zkratem pro DC stepdown měniče zajišťuje větraná krabička s otvory pro možnou regulaci napětí a proudu šroubovákem. Také je díky instalačním otvorům přišroubovaná vruty do konstrukční MDF desky.



Obrázek 16 - 3D model krabičky pro step-down měniče



Obrázek 17 – Finální podoba rozmístění komponent

6 Závěr – zhodnocení výsledků

V rámci projektu byl úspěšně realizován kompletní systém LED reaktivní osvětlení robota klavíristy s online řízením pomocí Raspberry Pi, propojený s robotem KUKA přes knihovnu OpenShowVar a podporující interaktivní režimy. Podařilo se sjednotit komunikaci mezi jednotlivými softwarovými a hardwarovými komponentami, řešit časově kritické úlohy v C++ a zajistit dostatečně robustní přenos informací s minimálním zpožděním.

Při realizaci byly identifikovány problémy spojené s délkou datového vedení, napěťovými úbytky a nutností převodu logických úrovní. Tyto technické komplikace byly úspěšně vyřešeny použitím level shifteru, samostatných napájecích větví a vhodným uložením elektrických komponent. Z hlediska mechanického návrhu byly vytvořeny vlastní 3D tištěné díly, které umožnily instalaci LED pásků bez zásahu do hlavní konstrukce. Pro vývoj, debug a sdílení souborů v rámci týmu jsme použili GitHub, kam jsme nahrávali naše změny a postupně dále rozvíjeli naše řešení.

Projekt tak přinesl nejen funkční světelný systém, ale i ucelené praktické řešení využitelné pro další rozšíření funkcionalit robota.

7 Zdroje

- [1] BÁTŘLA, Adam. *Návrh a realizace systému pro ovládání kláves pomocí robotického ramene*. Online, Bakalářská práce. Ostrava: Vysoká škola báňská – Technická univerzita Ostrava, 2022. Dostupné z: <http://hdl.handle.net/10084/147406>. [cit. 2025-12-07].
- [2] SRO, P. K. S. (n.d.). Převodník log.úrovně IIC/I2C 5V na 3,3V : H A D E X , spol. s r.o. 2011, Softima s.r.o. <https://www.hadex.cz/m436-prevodnik-logurovne-iici2c-5v-na-33v/>
- [3] Lukijanecek. (n.d.). *GitHub - Lukijanecek/AdvancedPianistRobot: PRII + OSRC + MITR*. GitHub. <https://github.com/Lukijanecek/AdvancedPianistRobot>